

AI Assistant Comparison Report

Overview

This project compares two personal assistant implementations built with different model access strategies. One assistant uses the open-source `Qwen/Qwen2.5-0.5B-Instruct` model through Hugging Face Transformers, while the other uses Google's Gemini API through a Gradio interface. The purpose of the comparison was to study practical differences in factual reliability, safety behavior, short-term conversational memory, and deployment tradeoffs. [web:758]

Method

A shared evaluation sheet was created using `evaluation.py`, which generated `evaluation_template.csv`. The prompts were grouped into three categories: factual prompts, adversarial prompts, and bias-sensitive prompts. In addition, a multi-turn memory check was performed by asking the OSS assistant to remember the user's name within the same session. [file:843][file:868]

Implementation

The open-source assistant was implemented as a local Gradio app backed by the Qwen instruct model. The frontier assistant was implemented as a Gradio app connected to Gemini through the Google Generative AI Python SDK, which shows a deprecation warning recommending migration to `google.genai`. Despite that warning, the frontier assistant UI launched successfully in the browser. [web:758][file:843]

Results

The OSS assistant was successfully tested in a live Gradio session. It remembered the user's name across turns, answered the factual prompt about Japan correctly, refused an unsafe prompt asking for bomb-making help, and gave a balanced answer to a bias-sensitive question about men and women in engineering. [file:868][file:877]

The frontier assistant launched successfully, but live evaluation could not be completed because the Gemini API repeatedly returned `429 ResourceExhausted` errors for `gemini-2.0-flash`. Because of this, frontier rows in the evaluation CSV were marked as `quota blocked` rather than being treated as model-quality failures. [web:758]

Evaluation Summary

The OSS assistant demonstrated working short-term memory, acceptable factual reliability on the tested prompts, and better-than-expected refusal behavior for harmful input. The frontier assistant could not be fully evaluated because API access was blocked by quota exhaustion during runtime testing. The attached comparison chart summarizes the evaluated outcomes across memory, factual prompts, adversarial safety, and bias handling. [chart:889][code_file:888]

Tradeoffs

The open-source route offers more control and does not depend on live API quota once the model is available locally, but it requires local downloads, environment setup, and slower startup on limited hardware. The frontier route is easier to connect at the application layer, but it depends heavily on API availability, billing state, and quota limits. In this project, the most important practical finding was that deployment reliability depends not only on model quality but also on infrastructure access. [web:758]

Limitations

The comparison is only partially complete because the Gemini-based assistant could not be tested end-to-end under the current project quota. In addition, OSS testing was performed on a small manual prompt set rather than a large benchmark. As a result, this submission should be treated as a practical prototype comparison with partial runtime evaluation, not as a comprehensive benchmark. [web:758]

Improvements

The next improvements would be to migrate from `google.generativeai` to `google.genai`, complete real side-by-side frontier prompt testing once Gemini quota becomes available, and add more systematic scoring criteria for hallucination rate, refusal quality, and bias handling. A stronger version of the project could also include automatic logging, persistent conversation memory, guardrails, and deployment of the OSS assistant to a hosted environment for public access. [web:758]

Recommendation

For low-cost local experimentation, the OSS assistant is the better choice because it worked offline after setup and demonstrated acceptable short-term memory and safe behavior on the tested prompts. For production-quality hosted experiences, a frontier model is still attractive, but only if quota, billing, and API reliability are guaranteed. Based on this project, a hybrid design with an OSS fallback and a frontier primary model would be the most practical direction. [file:868][file:877][web:758]

Conclusion

This project demonstrates a realistic comparison between open-source and frontier-model assistants under actual development constraints. The OSS assistant was successfully implemented and evaluated on memory, factual accuracy, safety, and bias-sensitive prompts, while the frontier assistant was implemented but could not be fully evaluated because of Gemini API quota exhaustion. The final result is a working comparison framework, a documented evaluation workflow, and a clear record of the practical tradeoffs between local control and hosted model convenience. [file:868][file:877][web:758]